# altairsim — CP/M 3 on an S100Computers Dual SD board

2026-08-16 · 84fc886

```
altairsim dualsd.toml

->                       (the V2 Z80 board's MASTER V6.6 monitor)
I

CP/M V3.0 Loader
64K CP/M VERSION (Non banked)
A: & B: = SD cards

A>DIR
```

John Monahan's **S100Computers** "**Dual SD**" **board** presents two microSD cards to an S-100 machine as raw 512-byte-sector block devices, and boots **CP/M 3**. An onboard ESP32-S3 runs the card firmware; the Z80 talks to it through two I/O ports at `80h` / `81h` with a `33h`-lead command protocol. See `docs/boards/dualsd.md` and `reference/dual-sd-card.md`.

**The monitor is the boot command.** There is no `BOOT` verb. The **V2 Z80 CPU board** (`v2z80`) carries the **MASTER V6.6** monitor in its paged EEPROM at `F000`–`FFFF`; `startup = ["RUN F000"]` cold-starts it exactly as EXAMINE `F000` + RUN did on the real board. At the monitor's `->` prompt, the `I` command reads the CP/M 3 cold loader off Dual SD drive 1 and jumps into the standard five-stage CP/M 3 cold boot.

**One card, one drive.** Each SD card holds a single CP/M drive, and the BIOS assigns the letter from the **socket** — socket 1 is `A:`, socket 2 is `B:` — not from anything on the card (move the same card to the other socket and it becomes `B:`).

**Two cards, on purpose.** The CP/M 3 BIOS cold-boot checks the STATUS-port card-detect line for *both* sockets and refuses to come up unless it sees a card in each — a single card boots the loader but then stops at `NO CCP.COM FILE`. So `cpm3-sd.img` is the bootable system (drive `A:`) and `blank.img` is a **blank spare** (drive `B:`): `DIR B:` reports `No File` until you copy files onto it or format it, and its backing file grows on write. (This presence gate was reverse-engineered from the shipping BIOS binary; the walk-through is in `reference/dual-sd-card.md` §5.1.)

`^E` (ATTN) takes the keyboard back to the simulator's monitor at any point; `RUN` resumes.

**Moving files to and from your host.** This machine fits the **host bridge** at ports `B0` / `B1`, and the boot card carries its three utilities: `HDIR` lists your host directory, `R <file>` reads a host file

onto the CP/M drive, and `W <file>` writes a CP/M file back out. They are sandboxed to the `hostdir` set in `dualsd.toml` (empty = the directory you launched `altairsim` from). See `docs/boards/hostbridge.md`. The machine also folds your terminal's Delete key to Backspace (`[console] bsdel = "bs"`) so editing at the `->` monitor and the `A>` prompt just works.

## A card is an image plus a `.geo` sidecar

Each socket mounts a raw card image (`cpm3-sd.img`) that has a sibling geometry descriptor of the same base name (`cpm3-sd.geo`) declaring the card's true size:

```
sector_size 512
sectors     769920
```

The sidecar declares the full **769920-sector** (≈394 MB) card, but `cpm3-sd.img` is only **~2 MB** — the CP/M 3 system, its directory, and its files. Every sector past the end of the image reads back the **erased-card fill byte (`0xFF`)**, which is what an unwritten microSD sector returns and all CP/M ever sees out there (free space). That is why a card that is hundreds of megabytes on real hardware ships here as a couple of megabytes, and why you can back up or swap a whole card as one host file. (Mounting a `.img` with no `.geo` beside it is an error — a card image is meaningless without its declared geometry.)

## The files

| File | What it is |
|---|---|
| `dualsd.toml` | The machine: V2 Z80 CPU + MASTER EEPROM (`v2z80`), a `propio` console, the `dualsd` board at `80h`/`81h` with a card in each socket, a front panel, the host bridge at `B0`/`B1`, and 64K RAM. |
| `cpm3-sd.img` + `cpm3-sd.geo` | Drive `A:` — the boot card: the CP/M 3 system image and its geometry sidecar. |
| `blank.img` + `blank.geo` | Drive `B:` — a blank spare card (`DIR B:` shows `No File`). |

**There is no undo.** The cards are mounted read/write because that is what a real board is, and CP/M writes to them. In a clone `git checkout` puts the images back; in a package you were handed, nothing does. Copy them first if you are about to test writes in anger, or add `readonly = true` to a drive in `dualsd.toml`.

This CP/M 3 system image comes from the S100Computers "CP/M Card Images" collection (DR-supplied CP/M 3, hobbyist/educational). The trailing sectors of the original card carried an unrelated leftover pattern that CP/M never reads; only the live filesystem is kept here.